QuizArena - Complete Architecture Brief

Document purpose: Explain the full system to management - product, architecture, database, realtime, security, and deployment.
Audience: Engineering managers, product stakeholders, technical reviewers
Date: August 2026
Codebase: QuizArena (modular monolith)

---

1. Executive Summary

QuizArena is a real-time live quiz platform. A host builds quizzes and runs a live session; participants join with a room code or QR (no account required); a display screen shows the session for the room audience.

The product is designed for education, corporate events, and entertainment. One host can run one active hosting room at a time. Sessions are synchronized in near real time: when the host starts a question, every participant and the display screen receive the same question, timer, and (later) reveal and leaderboard.

Concern | Choice
Style | Modular monolith (one backend process)
Frontend | Single React SPA with three role-based experiences
Backend | FastAPI (REST + native WebSockets)
Database | PostgreSQL (Neon in production); SQLite for local/tests
Hosting | Vercel (SPA) + Render (API/WS) + Neon (DB)

Why this architecture: Keeps delivery fast for a v1 product, keeps scoring and timers server-authoritative (fair play), and avoids complexity of microservices while the concurrent load target remains modest (~5 rooms x ~100 participants).

---

2. Product Overview

2.1 What the product does

1. Host signs up / logs in
2. Host creates a quiz (sections -> questions -> options), optionally adds media
3. Host validates quiz -> status becomes Ready
4. Host creates a live room from that quiz (structure is snapshotted)
5. Host opens lobby -> participants join via code/QR
6. Host starts quiz -> questions advance (automatic timer or manual)
7. Participants submit answers; scores and leaderboard update live
8. Quiz completes -> podium / final results
9. Host can export results and close the room

2.2 User roles

Role | Who | Auth | Main capabilities
Host | Quiz creator / session controller | JWT (email/username + password) | Build quizzes, control rooms, view participants, export results
Participant | Player | Room-scoped session token (no permanent account) | Join, answer, see personal score & leaderboard
Display | Presentation / projector screen | Secret token in URL | Read-only live view (question, timer, reveal, podium)

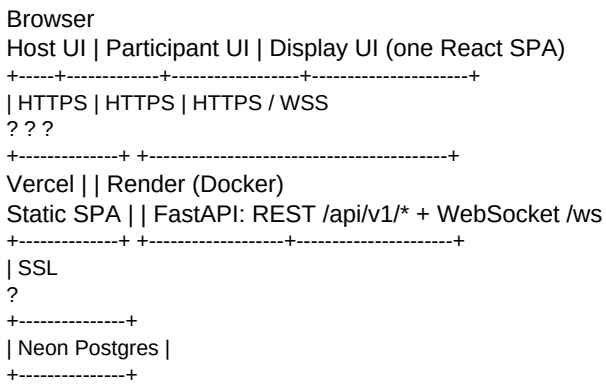
2.3 Capacity (v1 design targets)

- ~5 concurrent rooms
- ~100 participants per room

- <= ~1 second P95 realtime latency for critical events
- <= ~100 questions per quiz
- One hosting room per host at a time (Setup / Lobby / Active / Paused / SectionBreak)

---

3. High-Level System Architecture



Data flow principles

1. Server is the source of truth for room state, question state, timers, scores.
2. Clients submit actions (REST or WebSocket) and render events.
3. React Query caches REST data (quiz library, room lists, dashboard).
4. WebSocket + local reducers own the live hot path (question, timer, submissions, presence).
5. After host REST mutations, the frontend updates caches immediately so controls do not require a page refresh.

---

4. Technology Stack

Layer	Technologies
Frontend	React 19, TypeScript, Vite, TanStack Query, Axios, React Router, Tailwind CSS, Zod, Vitest
Backend	FastAPI, Uvicorn/Gunicorn, SQLAlchemy 2, Alembic, Pydantic, python-jose (JWT), bcrypt
Realtime	Native WebSockets (/ws) - not Socket.IO
Database	PostgreSQL (Neon) / SQLite (dev & tests)
Media	Local disk on Render (/app/storage); pluggable storage backend
Deploy	Neon + Render + Vercel; Docker for backend

---

5. Repository Layout

```
QuizArena/
+-- frontend/ React SPA (host + participant + display)
+-- backend/
+-- app/
| +-- api/routers/ REST endpoints
| +-- api/websocket/ /ws handler, auth, fan-out
| +-- models/ ORM tables
| +-- services/ Business logic + state machines
| +-- repositories/ Data access
| +-- core/ Security, middleware, rate limits
+-- alembic/versions/ Database migrations
+-- tests/ Unit + integration tests
+-- docs/ Architecture, schema, API, deploy guides
+-- render.yaml Render Blueprint
```

+-- docker-compose*.yml

---

6. Database Architecture (Critical)

6.1 Design idea: templates vs live snapshots

QuizArena separates reusable content from live session data:

Layer | Tables | Purpose
Template | quizzes, sections, questions, answer_options, quiz_configs | Editable quiz library
Session | live_rooms, session_sections, session_questions, session_options, room_configs | Immutable copy for one live run
Play | participants, responses, room_bans | Who joined and what they answered
Platform | admins, media_files, platform_settings, security_logs | Hosts, files, branding, audit

When a live room is created, the backend copies the quiz structure into session tables. Later edits to the quiz template do not change an already-created room. While a room is hosting, the quiz status becomes InUse.

6.2 Entity-relationship overview

admins --owns--? quizzes --? quiz_configs
|
+-- sections --? questions --? answer_options
+-- media_files
+-- live_rooms --? room_configs
|
+-- session_sections --? session_questions --? session_options
+-- participants --? responses
+-- room_bans

6.3 Table reference

admins (hosts)
- id, username (unique), email (unique, optional), name, password_hash, role (admin), timestamps
- Owns quizzes via quizzes.owner_id

quizzes
- id, owner_id, title, description, status, optional branding media
- Status values: Draft | Ready | InUse | Archived | Deleted

quiz_configs (1:1 with quiz)
- Question advance mode: manual | automatic
- Answer reveal: after_each | session_end
- Time bonus / streak bonus flags and point values
- Shuffle options / sections flags

sections / questions / answer_options
- Ordered sections under a quiz
- Questions: type Text | Image | Audio | Buzzer, prompt, explanation, points, time limit, multi-correct flag, optional media
- Options: text, is_correct, sort order

live_rooms
- Links to quiz; holds room lifecycle state
- room_code (6 characters, unique) for join
- secret_token (64 chars) for display URL
- quiz_title_snapshot, current_question_index
- Timestamps: started / paused / completed / closed
- pause_accumulated_ms for fair timers
- States: Setup | Lobby | Active | Paused | SectionBreak | Completed | Closed
- Lobby sub-state while in Lobby: LobbyOpen | LobbyClosed

room_configs
- Snapshot of quiz config at room creation (session scoring rules)

session_sections / session_questions / session_options
- Frozen copy of content for this room
- Session question state: Pending | Open | BuzzerOpen | BuzzerLocked | Closed | Revealed | Scored
- opened_at used for server-authoritative timers

participants
- Per room: display name + email (unique together in a room)
- session_token (unique) for auth
- State, connection status, score, streak, rank, answer counters

responses
- One row per (participant, session question)
- Selected option IDs (JSON), correctness, unanswered flag
- Base / time / streak / total points, submit time, response time ms, scored_at

room_bans
- Email banned from rejoining a room

media_files
- Storage key, MIME type, size, category, optional quiz link

platform_settings
- Platform name / logo

security_logs
- Login success / logout / login failed (audit)

6.4 Migrations

Migrations run automatically on Render container start (alembic upgrade head).

Migration | Purpose
Initial schema | Core tables
Response submit fields | response_time_ms, status
Scoring fields | scored_at, participant counters
Quiz builder | Explanation + builder support
opened_at | Authoritative question timers
Pause columns | Pause/resume timing
Host accounts | Admin email/name + quizzes.owner_id

---

7. Room & Quiz Lifecycle (State Machines)

7.1 Quiz template lifecycle

Draft --validate--? Ready --create room--? InUse
? |
+-- restore ?-- Archived Deleted (soft)

7.2 Live room lifecycle

Setup
| open lobby
?
Lobby --start--? Active ?--resume-- Paused
| | ? |
| | +-- pause ------+
| +-- section_break --? SectionBreak --continue--? Active
| +-- end --? Completed --close--? Closed

+-- close ------------------------------? Closed

Host UI implications

- Open Lobby only in Setup
- Start Quiz only in Lobby
- Pause only in Active; Resume only in Paused
- End Quiz in Active / Paused / SectionBreak
- Close Room in Completed (or Lobby in some list actions)

7.3 Question lifecycle (inside Active)

Pending --present--? Open --close--? Closed --reveal--? Revealed --score--? Scored
|
+-- (or Closed --mark_scored--? Scored)

Typical automatic path: Open -> (timer ends or all answered) -> Closed -> Revealed -> Scored -> next question / section break / quiz complete.

---

8. Authentication & Authorization

8.1 Host (JWT)

- Register / login with username or email + password
- Passwords hashed with bcrypt (policy: strong password rules)
- JWT HS256, Bearer header, typical 8-hour expiry
- Claims include admin id and role
- Used for all /api/v1 host routes and admin WebSocket connections

8.2 Participant (session token)

- Created on POST /api/v1/join with room code + display name + email
- High-entropy token stored on participants.session_token
- Used for participant REST (/participants/me, reconnect, leave) and WebSocket
- Room-scoped; no global user account

8.3 Display (secret URL)

- secret_token generated at room creation
- URL: /display/{secretToken}
- WebSocket: role=display&token=...
- Receive-only - cannot submit answers or control the room

8.4 Authorization rules (summary)

- Hosts only see/manage their own quizzes and rooms (owner_id)
- Participants can only act for their own participant id / room
- Display cannot send control events
- Emails are not exposed on public leaderboards

---

9. REST API Surface

Base path: /api/v1

Standard success envelope: { data, meta: { requestId } }
Errors: { error: { code, message, details }, meta }

Area | Examples
Health | /live, /ready, /health, /metrics

Auth | /admin/login, /register, /me, /logout, /change-password
Quizzes | CRUD, validate, archive, restore
Sections / questions / options | Nested CRUD under quiz
Media | Upload, list, stream, attach, delete
Live rooms | Create, list, get, config patch, open-lobby, start, pause, resume, end, close, delete, participants, results, CSV export
Join | POST /join
Participants | /participants/me, reconnect, leave
Dashboard | GET /dashboard/summary

WebSocket lives at /ws (not under /api/v1).

---

10. Realtime / WebSocket Architecture

10.1 Connection model

- Endpoint: /ws?role=admin|participant|display&token=...&roomId=... (roomId required for admin)
- On connect: authenticate -> connection:ack -> resync snapshot of current room
- In-memory connection pools per room (one admin, one display, many participants)
- v1 constraint: single Render instance (fan-out is process-local)

10.2 Important server events

Category | Events
Room | lobbyOpened, sessionStarted, paused, resumed, completed, closed, state_changed
Questions | section:started/break/continued, question:started/closed/reveal, quiz:completed
Answers | answer:accepted, rejected, received, submission_count
Scoring | question:scored, leaderboard:updated, score:personal
Presence | participant:joined, reconnected, disconnected

10.3 Important client events

Role | Events
Host | open_lobby, start_session, pause, resume, end, skip, start/close/reveal question, next, end_quiz
Participant | answer:submit
Display | none (receive-only)

10.4 Timers

- Server sets absolute timerEndsAt when a question opens
- Clients countdown locally
- Pause freezes remaining time; resume publishes a new end time
- Automatic mode: scheduler closes -> reveals -> dwells -> advances

10.5 Frontend sync (why refresh used to be needed)

Historically, host UI sometimes preferred a stale WebSocket room snapshot over a successful REST response (e.g. Open Lobby succeeded in DB but Start Quiz stayed disabled until refresh).

Current approach

1. Prefer the freshest lifecycle state between live WS and React Query
2. After every room mutation: setQueryData + invalidate related queries (list, detail, participants, results, dashboard)
3. Broadcast lobby open / close / end / close over WebSocket so all clients update
4. Participant completion forces Completed and navigates to results/podium without refresh

---

11. Scoring, Leaderboard & Auto-Progression

11.1 Scoring rules

- Scoring happens after reveal, never while the question is Open
- Idempotent (re-scoring does not double-count)
- Correctness: selected option set must exactly match correct set (multi-select is all-or-nothing)
- Points = base points (if correct) + optional time bonus + optional streak bonus
- Missing answers become unanswered records

11.2 Leaderboard

- Competition ranking across participants
- Broadcast as leaderboard:updated
- Final podium (top 3) included on quiz completion and available on reconnect resync

11.3 Auto-progression

- One background pipeline per room
- On question start (automatic mode): wait until timer end (or all answered) -> close -> reveal -> short dwell -> next
- Cancelled on pause/end; resumed on resume

---

12. Frontend Architecture

12.1 Route map (conceptual)

Experience | Routes (examples)
Public landing | /
Host auth | /host/login, /host/signup
Host app | /admin/dashboard, quizzes, live rooms, room monitor, results
Participant | /join, lobby, quiz play, results
Display | /display/:secretToken

12.2 State ownership

Data | Owner
Quiz library, dashboard stats, room list/detail | TanStack Query
Live question / timer / submissions / presence | WebSocket reducers
Participant identity | sessionStorage + context
Host JWT | auth token storage

12.3 Key frontend modules

- useLiveRoomMutations + syncLiveRoomCaches - keep REST caches fresh
- useAdminWebSocket / useParticipantWebSocket / display reducer - live session
- preferFreshestRoom / canRest - host control enablement

---

13. Deployment Architecture

13.1 Topology

Layer | Platform | Role
Database | Neon | Managed PostgreSQL
Backend | Render (Docker) | REST + WebSockets, always-on plan
Frontend | Vercel | Static SPA

Important: Free Render instances sleep and drop WebSockets - use Starter or higher.

13.2 Critical environment variables

Backend: DATABASE_URL, JWT_SECRET_KEY, PUBLIC_APP_URL, CORS_ORIGINS, TRUSTED_HOSTS, admin seed vars, storage pat

Frontend: VITE_API_BASE_URL, VITE_WS_BASE_URL

13.3 Operational notes

- Migrations run on container start
- Media stored on Render persistent disk at /app/storage
- Health check: /api/v1/ready
- Single instance for WebSocket fan-out in v1

---

14. Security Overview

Area | Approach
Transport | HTTPS / WSS
Host passwords | bcrypt + complexity policy
Host sessions | JWT Bearer (not cookies)
Participant / display | High-entropy room-scoped tokens
Input validation | Pydantic (API) + Zod (UI)
SQL | Parameterized ORM queries
Uploads | MIME/size checks; opaque storage keys
Rate limits | Login & join throttles
Headers | CORS allowlist, TrustedHost, CSP/HSTS/nosniff
Audit | Security logs for login events
Secrets | Environment only; never logged

Known v1 limits (honest for management):

- WebSocket state is in-process -> horizontal scale needs sticky sessions or a shared pub/sub later
- Display URL is a bearer secret (anyone with the link can watch)
- Rate limits are in-memory per instance

---

15. Testing Strategy

Backend (pytest)

- Unit: security, dashboard aggregates, bootstrap admin, settings
- Integration: auth, quiz builder, live rooms, join, answer submit, scoring, execution, auto-progression, WebSockets, full realtime lifecycle

Frontend (Vitest)

- Live reducers (participant / display)
- Room lifecycle helpers (button enablement / freshest state)
- Join / session / UI smoke tests

Manual smoke (post-deploy)

Login -> build quiz -> create room -> open lobby -> join from phone -> start -> answer -> pause/resume -> complete -> podium -> export

---

16. End-to-End Story (How to Demo)

1. Host registers and logs in

2. Creates quiz with one section and several questions
3. Validates quiz -> Ready
4. Creates live room -> Setup
5. Opens lobby -> Lobby (Start Quiz enables immediately)
6. Participant joins with room code; host count increments live
7. Host starts quiz -> Question 1 appears for participant + display; timer runs
8. Participant submits -> submission count increments; after scoring, leaderboard updates
9. Quiz completes -> participant sees podium/results; host End Quiz disabled because room is Completed
10. Host closes room; quiz returns toward Ready for reuse

No page refresh is required for controls, counts, leaderboard, or podium when the realtime path is healthy.

---

17. Glossary

Term | Meaning
Host | Authenticated quiz operator (admin account)
Live room | One running instance of a quiz session
Snapshot | Frozen copy of quiz content for a room
Resync | WebSocket snapshot sent on connect/reconnect
Podium | Top-3 final standing
Display | Read-only presentation client
InUse | Quiz locked while a hosting room exists

---

18. Appendix - Key Document Paths

Topic | Path
Product / SRS | docs/PROJECT_SPEC.md
System architecture | docs/SYSTEM_ARCHITECTURE.md
Database schema | docs/DATABASE_SCHEMA.md
API | docs/API.md, docs/API_SPEC.md
WebSocket events | docs/SOCKET_EVENTS.md
Deploy guide | docs/DEPLOY_RENDER_VERCEL.md
Env vars | docs/EnvironmentVariables.md

---

19. One-Slide Talking Points for Your Manager

1. What: Live quiz platform - host builds content, players join by code, big screen syncs.
2. How built: One FastAPI backend + one React app; Postgres; WebSockets for live sync.
3. Fairness: Server owns timers and scores; clients only display and submit.
4. Data model: Editable quiz templates + immutable session snapshots per live room.
5. Scale (v1): Single always-on API instance; suitable for classroom / event sized load.
6. Deploy: Neon DB, Render API/WS, Vercel frontend.
7. Security: JWT hosts, room tokens for players, bcrypt, rate limits, CORS, audit logs.
8. Status: End-to-end live flow works without refresh when deployed with current main branch.

End of architecture brief.